

1. Elementy podstawowe

Uczestnicy (lifelines):

- Umieszczani na górze diagramu jako prostokąty, z pionową przerywaną linią biegnącą w dół
- Mogą to być aktorzy (ludzie/systemy zewnętrzne inicjujące akcję) lub obiekty (instancje klas, np. `:System`, `:BazaDanych`)
- Nazwa obiektu z dwukropkiem przed nazwą klasy oznacza konkretną instancję, nie samą klasę (`:System` = instancja klasy `System`)
- Kolejność uczestników od lewej do prawej powinna odzwierciedlać naturalny przepływ interakcji — inicjator zwykle po lewej

Oś czasu:

- Czas płynie wyłącznie od góry do dołu — to najważniejsza zasada tego typu diagramu
- Nie ma cofania się w czasie; jeśli coś się powtarza, rysuje się to jako kolejne wywołanie niżej, a nie jako strzałkę wracającą w górę

2. Komunikaty (messages)

- Strzałka ciągła z pełnym grotem = komunikat synchroniczny (wywołanie metody) — nadawca czeka na zakończenie
- Strzałka przerywana = komunikat zwrotny (return) — pokazuje wynik/odpowiedź
- Każdy komunikat powinien mieć etykietę w formie `nazwaMetody(parametry)`, najlepiej zgodną z nazewnictwem z Class Diagram
- Strzałka zawsze biegnie poziomo między dwiema liniami życia (nigdy po skosie)
- Kolejność komunikatów od góry do dołu = kolejność wykonania w czasie

3. Pasy aktywacji (activation bars)

- Cienki prostokąt na linii życia pokazuje, że obiekt jest aktywny (wykonuje operację)
- Zaczyna się w momencie otrzymania komunikatu, kończy w momencie zwrócenia wyniku (lub zakończenia przetwarzania)
- Pasy mogą się zagnieżdżać, jeśli obiekt A wywołuje obiekt B, będąc wciąż aktywnym

4. Zasady dobrej praktyki

- Jeden diagram = jeden konkretny scenariusz (zwykle główny przebieg jednego przypadku użycia — dokładnie tak, jak zrobiliśmy: osobny diagram dla każdego use case)
- Nie przeciążaj diagramu wszystkimi możliwymi ścieżkami alternatywnymi naraz — ścieżki błędów/wyjątków lepiej pokazać w osobnym, uproszczonym diagramie albo pominąć, jeśli nie są kluczowe
- Liczba uczestników powinna być ograniczona (zwykle 3–6) — więcej robi diagram nieczytelny

- Komunikaty powinny być na poziomie logiki biznesowej, nie szczegółów technicznych (np. `sprawdźDostępność()`, a nie `SELECT * FROM ksiazki WHERE ...`)
- Jeśli chcesz pokazać warunek/alternatywę, w pełnym UML używa się ramki **alt** (alternative) lub **opt** (optional) obejmującej fragment diagramu — to bardziej zaawansowana notacja, przydatna, gdy jeden diagram ma pokazać też ścieżkę błędu

5. Częsty błąd, którego unikaliśmy

Mieszanie diagramu sekwencji z diagramem aktywności — sekwencja pokazuje kto z kim się komunikuje i w jakiej kolejności, a nie logikę warunkową (to rola Activity Diagram). Dlatego np. w naszych diagramach decyzje typu "czy dostępny?" nie pojawiają się jako diamenty, tylko są uproszczone do jednej ścieżki głównego przebiegu.